WELT 0 · DOPPELSTUNDE 3 · LEVEL 0.3

W0·3 – Die Vorratskammer: Reihungen und Zeichenketten

Datei: `AQ_W0_Akademie_Geruest.txt` (Klasse Vorrat) · IDE-Level 0.3 · 🎯 Im Lab: **Level 0.3**

Unter der Werkstatt liegt die **Vorratskammer** der Akademie: lange Regale mit nummerierten Fächern voller Kräuter, Kristalle und Tränke. Der Kammermeister führt über jedes Regal Buch – nicht mit hundert einzelnen Variablen, sondern mit einer **Reihung**: ein Name, viele Fächer, jedes mit seiner Nummer.

Reihungen stecken in fast jeder Abituraufgabe. Wer hier sauber mit Index, Länge und Schleifengrenzen umgeht, verschenkt später keine BE. Und weil auf den Etiketten Text steht, lernst du auch, welche **Zeichenketten-Operationen** im Abitur erlaubt sind.

Infoteil 1: Reihungen anlegen, lesen, schreiben

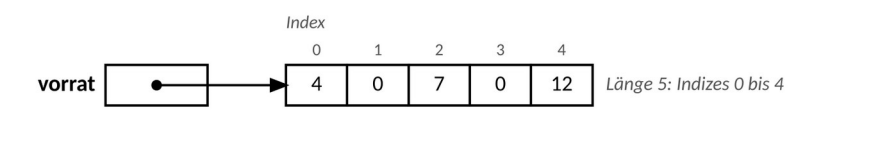


Abbildung 1: Die Variable vorrat verweist auf eine Reihung mit fünf Fächern

- **Anlegen mit Werten:** `int[] vorrat = {4, 0, 7, 0, 12};` – die Länge (5) ergibt sich aus den Werten.
- **Anlegen mit Länge:** `int[] regal = new int[6];` – alle Fächer sind mit dem Standardwert gefüllt: 0 bei Ganzzahlen, 0.0 bei Kommazahlen, false bei Wahrheitswerten, `null` bei Zeichenketten und Objekten. Die Länge ist danach **fest**.
- **Zugriff über den Index:** `vorrat[2]` liefert 7, `vorrat[1] = 5;` schreibt in das zweite Fach. Der Index läuft von **0 bis Länge − 1**.
- **Länge:** `vorrat.length` (ohne Klammern!) – bei Zeichenketten dagegen `s.length()` mit Klammern. Das letzte Element ist `vorrat[vorrat.length - 1]`.
- **Ungültiger Index** (z. B. −1 oder 5): Das Programm bricht mit einer **ArrayIndexOutOfBoundsException** ab.

Java	Struktogramm (wie im Abitur)
`r.length`	Länge von r
`int[] neu = new int[n];`	neu ← neue Reihung von Ganzzahl der Länge n
`r[i] = r[i] + 1;`	r[i] ← r[i] + 1
`for (int i = 0; i < r.length; i++)`	für i von 0 bis Länge von r − 1
`for (int x : r)`	für jedes x in r

Aufgabe 0: Blitz-Warm-up: Fach für Fach

PAPIER ★

Gegeben sind `int[] r = {8, 3, 0, 5, 1};` und `String s = "Vorrat";`. Gib den Wert jedes Ausdrucks an. Führt ein Ausdruck zu einem Fehler, schreibe „Fehler“.

Ausdruck	`r.length`	`r[r.length - 1]`	`r[r[4]]`	`r[2] + r[3]`	`r[5]`
Wert					

Ausdruck	`s.length()`	`s.charAt(2)`	`s.substring(1, 4)`	`s.charAt(s.length() - 1)`	`(int) s.charAt(0)`
Wert					

Tipp: `'V'` hat den ASCII-Wert 86, `'A'` den Wert 65, `'a'` den Wert 97.

Infoteil 2: for oder for-each?

Beide Schleifen besuchen alle Elemente. Die **for-each-Schleife** liefert dir in `x` aber nur eine **Kopie des Werts** – keinen Index und keinen Zugriff auf das Fach selbst.

```
int summe = 0;
for (int i = 0; i < r.length; i++) {
    summe = summe + r[i];
}
```

```
int summe = 0;
for (int x : r) {
    summe = summe + x;
}
```

Du willst …	Zählschleife mit Index	for-each
alle Werte nur **lesen** (Summe, Zählen, Suchen nach Wert)	✓	✓
Werte **verändern** (`r[i] = …`)	✓	× – `x = …` ändert nur die Kopie
die **Position** wissen (Index liefern)	✓	×
mit **Nachbarn** vergleichen (`r[i - 1]`, `r[i + 1]`)	✓ (Grenzen anpassen!)	×
nur einen **Teil** durchlaufen oder rückwärts	✓	×

Aufgabe 1: Regalbau

PAPIER ★

Der Kammermeister baut ein neues Regal und befüllt es.

```
1  int[] regal = new int[6];
2  regal[0] = 5;
3  regal[regal.length - 1] = 2;
4  regal[2] = regal[0] + regal[5];
5  for (int i = 1; i < regal.length; i = i + 2) {
6      regal[i] = regal[i - 1] * 2;
7  }
```

a) Trage den Inhalt der Reihung nach jeder Anweisung ein. Für die Schleife: eine Zeile pro Durchlauf (mit dem Wert von i). **[4 BE]**

Zeitpunkt	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5
nach Zeile 1						
nach Zeile 2						
nach Zeile 3						
nach Zeile 4						
i = ____						
i = ____						
i = ____						

b) Ergänzt man die Anweisung `regal[6] = 1;`, bricht das Programm ab. Erläutere, warum. **[1 BE]**

c) Nach `String[] namen = new String[3];` wird `println(namen[0].length());` ausgeführt. Gib an, was in `namen[0]` steht und was passiert. **[2 BE]**

Aufgabe 2: for oder for-each?

PAPIER ★★

Für jede Aufgabe auf der Reihung `vorrat` soll eine Schleife geschrieben werden. Entscheide, welche Schleife geeignet ist, notiere den **Schleifenkopf** in Java und begründe kurz.

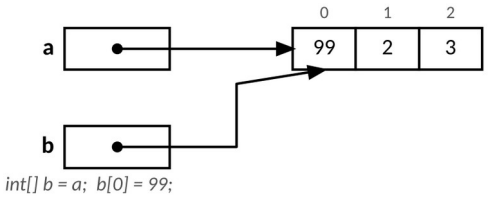
Nr.	Aufgabe	for / for-each	Schleifenkopf in Java	Begründung
1	Summe aller Vorräte bilden			
2	jeden Vorrat um 1 erhöhen			
3	Index des ersten leeren Fachs finden			
4	prüfen, ob ein Fach mehr als 10 enthält			
5	zählen, wie oft ein Fach mehr enthält als sein **rechter** Nachbar			

Bewertung: [5 BE]

Infoteil 3: Referenz oder Kopie?

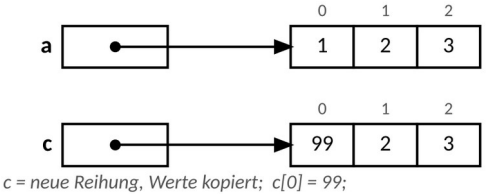
Eine Reihungsvariable enthält **nicht die Reihung selbst**, sondern einen **Verweis (Referenz)** auf sie – im Bild ein Pfeil. Das hat Folgen:

```
int[] a = {1, 2, 3};
int[] b = a;      // kopiert nur den Pfeil!
b[0] = 99;        // a[0] ist jetzt auch 99
```



int[] b = a; b[0] = 99;

```
int[] a = {1, 2, 3};
int[] c = new int[a.length];
for (int i = 0; i < a.length; i++) {
    c[i] = a[i];  // Wert für Wert kopieren
}
c[0] = 99;        // a bleibt {1, 2, 3}
```



c = neue Reihung, Werte kopiert; c[0] = 99;

- `int[] b = a;` erzeugt **keine** neue Reihung: a und b verweisen auf **dieselbe** Reihung. Jede Änderung über b ist auch über a sichtbar.
- Eine **echte Kopie** braucht `new` und eine Schleife, die jeden Wert einzeln überträgt.
- `a == b` vergleicht bei Reihungen die **Pfeile**, nicht die Inhalte. Zwei Reihungen mit gleichen Werten sind mit `==` verschieden.

Aufgabe 3: Das Regal-Rätsel

PAPIER ★★

Ein Lehrling hat mit drei Etiketten und zwei Regalen experimentiert. Überlege zuerst – starte nichts!

```
1  int[] links = {2, 4, 6};
2  int[] rechts = links;
3  int[] neu = {2, 4, 6};
4  rechts[1] = 40;
5  neu[2] = 60;
6  links = neu;
7  links[0] = 20;
```

a) Zeichne das Speicherbild nach Zeile 7: Variablen als Kästchen, Reihungen als Kästchenreihe, Referenzen als Pfeile. **[3 BE]**

b) Gib den Inhalt der drei Reihungen an, auf die links, rechts und neu nach Zeile 7 verweisen. **[2 BE]**

c) Gib die Werte von `links == neu` und `rechts == neu` an. Erläutere, warum `{2, 4, 6} == {2, 4, 6}` in diesem Sinn falsch wäre, und beschreibe, wie man zwei Reihungen auf **gleichen Inhalt** prüft. **[3 BE]**

Infoteil 4: Eine neue Reihung zurückgeben

Viele Abituraufgaben verlangen: „… liefert eine **neue** Reihung …; r bleibt unverändert.“ Das Muster hat immer drei Schritte:

1. Neue Reihung der passenden Länge anlegen: `int[] neu = new int[r.length];`
2. In einer Schleife aus r **lesen** und in neu **schreiben** – r selbst wird nie verändert.
3. Die neue Reihung zurückgeben: `return neu;`

```
public static int[] quadrate(int[] r) {
    int[] neu = new int[r.length];
    for (int i = 0; i < r.length; i++) {
        neu[i] = r[i] * r[i];
    }
    return neu;
}
// quadrate({3, -2, 5}) liefert {9, 4, 25}; die leere Reihung liefert eine leere Reihung
```

Achtung: `int[] neu = r;` ist **keine** neue Reihung (Infoteil 3) – dann würde r verändert.

Aufgabe 4: Ein Fach weiterrücken

PAPIER ★★★

Bei der Inventur rückt jeder Vorrat ein Fach nach rechts; der letzte wandert ganz nach vorn.

```
1  public static int[] verschoben(int[] r) {
2      int[] neu = new int[r.length];
3      for (int i = 0; i < r.length; i++) {
4          neu[(i + 1) % r.length] = r[i];
5      }
6      return neu;
7  }
```

a) Erstelle eine Tracetabelle für den Aufruf `verschoben(r)` mit `r = {7, 1, 5}` und gib die zurückgegebene Reihung an. **[3 BE]**

i	(i + 1) % r.length	r[i]	neu (nach dem Durchlauf)

b) Begründe, dass r unverändert bleibt. Erläutere, was sich ändern würde, wenn Zeile 2 `int[] neu = r;` lautete. **[2 BE]**

c) Für die leere Reihung wäre `% r.length` eine Division durch 0. Begründe, warum `verschoben` trotzdem ohne Fehler eine leere Reihung liefert. **[2 BE]**

d) Stelle die Methode als Struktogramm nach EH § 5 dar. **[3 BE]**

Infoteil 5: Zeichenketten – erlaubt oder verboten?

Im Abitur ist die Liste der Zeichenketten-Operationen **abschließend** (AB 1, Teil B, Infoteil 4). Alles andere musst du aus den erlaubten Operationen selbst bauen:

verboten (Beispiele)	So geht es mit erlaubten Operationen
`s.toLowerCase()`, `s.toUpperCase()`	beide Fälle prüfen: `c == 'a' || c == 'A'`; umwandeln über ASCII: `(char) (c + 32)` macht aus 'D' ein 'd'
`s.indexOf(t)`	Schleife über i mit `charAt(i)` bzw. `substring(i, i + t.length()).equals(t)`
`s.replace(…)`, `s.split(…)`, `s.trim()`	Schleife über die Zeichen, Ergebnis mit + zusammensetzen
`s.startsWith(t)`, `s.endsWith(t)`	erst die Längen prüfen, dann `substring(…)` und `equals`
`s.isEmpty()`	`s.length() == 0`
`Character.isDigit(c)` u. Ä.	Bereich prüfen: `c >= '0' && c <= '9'`

- **Zeichen** (`char`, einfache Anführungszeichen) vergleichst du mit `==`: `s.charAt(i) == 'e'`. **Zeichenketten** vergleichst du mit `equals`.
- `s.charAt(i) == "e"` kompiliert nicht: Zeichen und Zeichenkette sind verschiedene Typen.
- `s.compareTo(t)` ist negativ, wenn s im Wörterbuch (genauer: nach ASCII-Werten) **vor** t steht, 0 bei Gleichheit, sonst positiv. Großbuchstaben stehen vor Kleinbuchstaben.

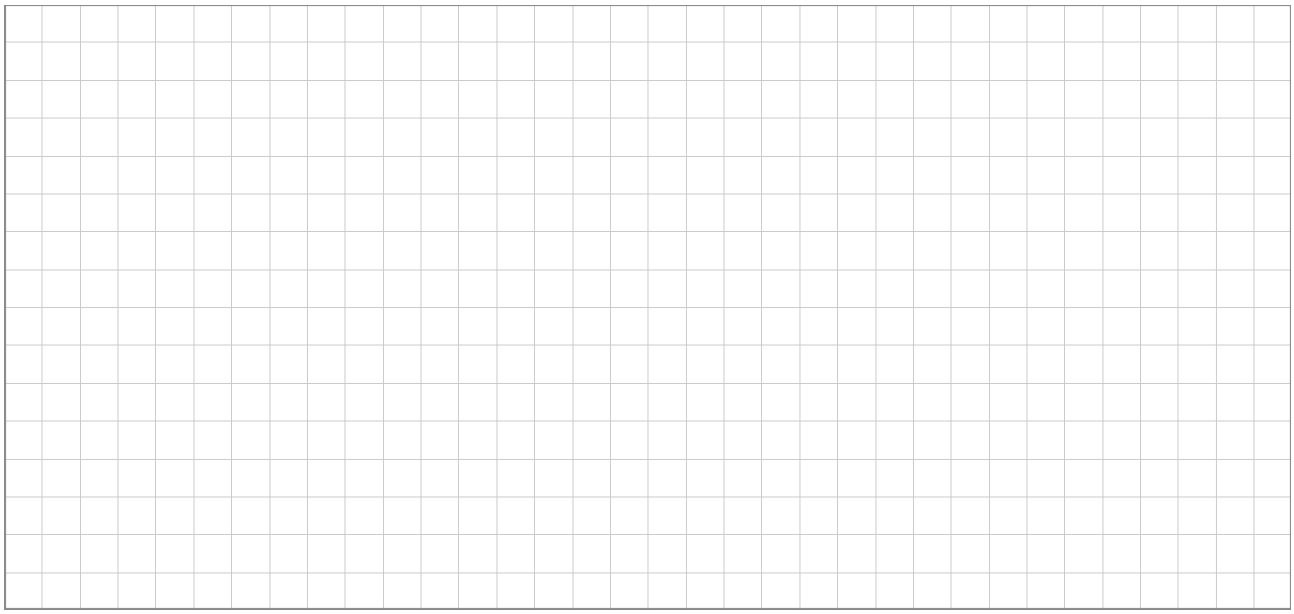
Aufgabe 5: Die verbotenen Etiketten

PAPIER ★★

Der Zauberlehrling hat drei Etiketten-Methoden mit **verbotenen** Operationen geschrieben:

```
public static boolean endetMit(String s, String ende) {
    return s.endsWith(ende);
}
public static int anzahlLeerzeichen(String s) {
    return s.split(" ").length - 1;
}
public static boolean istGrossbuchstabe(char c) {
    return Character.isUpperCase(c);
}
```

a) Schreibe alle drei Methoden so um, dass sie nur erlaubte Operationen verwenden. Achte bei `endetMit` darauf, dass `ende` länger als `s` sein kann. **[5 BE]**



b) Gib an, ob der Ausdruck negativ, 0 oder positiv ist. **[2 BE]**

Ausdruck	negativ, 0 oder positiv?	Ausdruck	negativ, 0 oder positiv?
`"Drache".compareTo("Drachen")`		`"Eis".compareTo("Feuer")`	
`"elf".compareTo("Elf")`		`"Rune".compareTo("Rune")`	

c) Gib den Wert an: `(int) 'a'` · `(char) ('a' + 2)` · `(char) ((int) 'D' + 32)` **[1 BE]**

🎯 Im Lab üben: Level 0.3 „Die Vorratskammer: Reihungen und Zeichenketten“ – Punkte gibt es dort automatisch

Passend zu den Infoteilen 1–5 und den Aufgaben 0–7: Indizes und Schleifengrenzen, Code-Puzzle, Lückencode und Bug-Hunt zu Reihungen und Zeichenketten – jede Antwort wird sofort geprüft. Danach geht es in der Online-IDE mit IDE-Level 0.3 weiter.

Aufgabe 6: Level 0.3 – Die Vorratskammer

IDE ★ – ★★★

a) Öffne deine Akademie-Datei aus DS 2. Heute arbeitest du in der Klasse **Vorrat**; Testklasse `L03VorratTest`.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	anzahlLeer(r: Reihung von Ganzzahl): Ganzzahl letzterWert(r: Reihung von Ganzzahl): Ganzzahl enthaelt(r: Reihung von Ganzzahl, x: Ganzzahl): Wahrheitswert	`basis_`
★★ Erweiterung	umgedreht(r: Reihung von Ganzzahl): Reihung von Ganzzahl verdopple(r: Reihung von Ganzzahl): nichts anzahlAnstiege(r: Reihung von Ganzzahl): Ganzzahl	`erweiterung_`
★★★ Experte	zaehleZeichen(s: Zeichenkette, c: Zeichen): Ganzzahl ohneVokale(s: Zeichenkette): Zeichenkette beginntMit(s: Zeichenkette, anfang: Zeichenkette): Wahrheitswert	`experte_`

b) **Vorhersage ohne Gerät:** Das Hauptprogramm enthält ein „Referenz-Rätsel“ (`int[] b = a; b[0] = 99;`) und gibt den Vorrat `{4, 0, 7, 0, 12, 9}` nach `Vorrat.verdopple(vorrat)` aus. Notiere beide Ausgaben, dann starte und vergleiche.

Tipps aus der Vorratskammer

- **Leere Reihung zuerst:** Wo du auf `r[0]` oder `r[r.length - 1]` zugreifst, frage vorher `r.length == 0` ab.
- **Suche:** `return false` gehört **hinter** die Schleife – nicht in einen else-Zweig.
- **verdopple:** Mit for-each änderst du nur Kopien (Infoteil 2).
- **Zeichenketten:** nur die erlaubten Operationen (Infoteil 5) – die Tests merken einen Verstoß nicht, das Abitur schon.

Aufgabe 7: Bug-Hunt in der Vorratskammer

PAPIER ★★

In jeder Methode steckt **genau ein** Fehler.

Methode A – soll true liefern, wenn r aufsteigend sortiert ist:

```
public static boolean istSortiert(int[] r) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] > r[i + 1]) {
            return false;
        }
    }
    return true;
}
```

Methode B – soll eine **neue** Reihung mit den dreifachen Werten liefern; r bleibt unverändert:

```
public static int[] verdreifacht(int[] r) {
    int[] neu = r;
    for (int i = 0; i < r.length; i++) {
        neu[i] = r[i] * 3;
    }
    return neu;
}
```

Methode C – soll zählen, wie oft e oder E in s vorkommt:

```
public static int anzahlE(String s) {
    int anzahl = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == "e" || s.charAt(i) == 'E') {
            anzahl++;
        }
    }
    return anzahl;
}
```

Methode D – soll true liefern, wenn a und b gleich lang sind und an jeder Stelle denselben Wert haben:

```
public static boolean gleicherInhalt(int[] a, int[] b) {
    return a == b;
}
```

a) Gib für jede Methode die fehlerhafte Zeile an und beschreibe den Fehler. **[4 BE]**

Methode	Zeile	Fehler
A		
B		
C		
D		

b) Gib eine Reihung an, für die Methode A ohne Absturz ein Ergebnis liefert, und eine, bei der sie abbricht. **[2 BE]**

c) Gib die Ausgabe an: `int[] b = {1, 2, 3}; int[] c = verdreifacht(b); println(b[0] + " " + c[0]);` **[1 BE]**

d) Korrigiere die Methoden A, B und C jeweils in einer Zeile. Implementiere Methode D korrekt. **[4 BE]**

Für Schnelle: Die Lotterie der Kammer

- **6 aus 49:** Fülle eine Reihung der Länge 6 mit Zufallszahlen von 1 bis 49 – **ohne** doppelte Zahlen. Welche Methode aus deiner Klasse Vorrat hilft beim Prüfen?
- **Der Reindrängler:** `eingefuegt(r: Reihung von Ganzzahl, pos: Ganzzahl, wert: Ganzzahl): Reihung von Ganzzahl` liefert eine neue, um 1 längere Reihung, in der `wert` an Index `pos` steht; alle Werte ab `pos` rücken nach hinten. Zeichne dir die beiden Reihungen zuerst untereinander und verbinde alte und neue Indizes mit Pfeilen.
- **Geheimschrift (Caesar):** `verschluessle(text: Zeichenkette, k: Ganzzahl): Zeichenkette` verschiebt jeden Kleinbuchstaben um k Stellen im Alphabet („xyz“ mit k = 3 wird „abc“), alle anderen Zeichen bleiben. Nur mit `charAt`, `+`, `(int)`, `(char)` und `%`. Knacke damit: „kdoor zhow“.

Side-Quest: freiwillig und ohne Punkte – stellt eure Ergebnisse im Team oder der Lehrkraft vor.